

Universidad Rafael Landívar.

Facultad de Ingeniería.

Ingeniería en Informática y Sistemas.

Manejo e Implementación de Archivos, Sección 02.

Docente: Luis Enrique Aguilar Rojas.

Actividad No. 1 Semana #13 - APIS Conceptos

Estudiante: Arrazola Escobar, Luna Samantha

Carné: 1022725

Guatemala, 1 de octubre de 2026.

Las APIs son herramientas que permiten que diferentes aplicaciones, sistemas o servicios puedan comunicarse entre sí. API significa Application Programming Interface o Interfaz de Programación de Aplicaciones. Por medio de ellas, un programa puede solicitar información o utilizar funciones de otro sistema sin necesidad de conocer cómo está desarrollado internamente.

A) Tipos de API

* APIs Públicas (Open APIs)

Son APIs que una empresa u organización pone a disposición de desarrolladores externos para que puedan utilizar ciertas funciones o datos de sus servicios. Generalmente cuentan con documentación que explica cómo utilizarlas y algunas requieren una clave de acceso.

Ejemplo real: Un banco puede utilizar una API pública para permitir que desarrolladores externos consulten información como tipos de cambio, ubicación de agencias o cajeros automáticos.

* APIs Internas (Private APIs)

Son utilizadas únicamente dentro de una empresa u organización. Permiten que sus diferentes aplicaciones y sistemas se comuniquen entre sí de manera controlada y sin exponer información al público.

Ejemplo real: Una universidad puede utilizar una API interna para comunicar su sistema de estudiantes con la plataforma de cursos y consultar datos como inscripciones, notas y horarios.

* APIs de Socios (Partner APIs)

Son APIs que se comparten únicamente con empresas o socios autorizados. Para utilizarlas normalmente se necesita tener un acuerdo con la organización y contar con credenciales de acceso.

Ejemplo real: Una tienda en línea puede proporcionar una API a una empresa de envíos para compartir información de los pedidos y consultar el estado de las entregas.

* APIs Compuestas (Composite APIs)

Permiten realizar varias operaciones relacionadas mediante una sola solicitud. De esta manera se evita tener que hacer una petición diferente para cada acción.

Ejemplo real: Al comprar en una tienda en línea, una API compuesta puede crear el pedido, registrar los productos comprados y actualizar el inventario mediante una sola solicitud.

B) Estilos arquitectónicos

* REST

REST es uno de los estilos más utilizados para desarrollar APIs. Funciona normalmente mediante HTTP y utiliza métodos como GET para consultar, POST para crear, PUT o PATCH para modificar y DELETE para eliminar información. Los datos suelen enviarse en formato JSON.

Uso: Es común en aplicaciones web, aplicaciones móviles, tiendas en línea y otros servicios que necesitan intercambiar información por Internet.

* GraphQL

GraphQL permite que el cliente indique exactamente qué información necesita obtener del servidor. Esto evita recibir datos adicionales que no serán utilizados y permite realizar consultas más específicas.

Uso: Se utiliza principalmente en aplicaciones que manejan grandes cantidades de información relacionada o necesitan controlar exactamente los datos que reciben.

* SOAP

SOAP es un protocolo para intercambiar información entre sistemas. Utiliza principalmente el formato XML y posee reglas más estrictas que otros estilos para organizar las solicitudes y respuestas.

Uso: Se utiliza en sistemas empresariales, servicios financieros y aplicaciones donde se necesita una comunicación estructurada y con mayores controles.

* gRPC

gRPC permite que diferentes servicios se comuniquen de manera rápida y eficiente. Utiliza comúnmente Protocol Buffers para definir la estructura de los datos que se intercambian.

Uso: Es común en sistemas distribuidos y arquitecturas de microservicios, especialmente cuando varios servicios necesitan comunicarse constantemente.

* WebSocket

WebSocket permite mantener una conexión abierta entre el cliente y el servidor. De esta manera, ambos pueden enviarse información en cualquier momento sin crear una nueva conexión para cada mensaje.

Uso: Se utiliza cuando se necesita comunicación en tiempo real, por ejemplo en chats, videojuegos en línea, notificaciones o sistemas de seguimiento en vivo.

* Webhooks

Los webhooks permiten que un sistema notifique automáticamente a otro cuando ocurre determinado evento. Así, el sistema receptor no necesita consultar constantemente si ocurrió algún cambio.

Uso: Son comunes en pagos y tiendas en línea. Por ejemplo, una plataforma de pagos puede enviar una notificación a una tienda cuando el pago de un cliente fue aprobado.

Conceptos relacionados

* Payload

El payload es la información principal que se envía dentro de una petición o respuesta de una API. Puede contener datos en formatos como JSON.

Ejemplo: Para registrar un estudiante se podría enviar:

```
{ "nombre": "Luna", "carne": "1022725" }
```

* Request

Un request es la petición que un cliente envía a un servidor para solicitar información o realizar alguna acción.

Ejemplo: Una aplicación puede realizar una petición GET para solicitar al servidor la información de un usuario.

* Response

Un response es la respuesta que el servidor devuelve después de procesar un request. Puede incluir la información solicitada y un código que indique el resultado de la operación.

Ejemplo: El código 200 OK indica que una solicitud se realizó correctamente, mientras que 404 Not Found indica que el recurso solicitado no fue encontrado.

* CURL

CURL es una herramienta que permite realizar solicitudes a servidores desde una terminal. Es útil para probar el funcionamiento de una API sin necesidad de crear una aplicación completa.

Ejemplo: `curl https://api.ejemplo.com/usuarios`

Este comando realiza una solicitud al servidor para obtener información del recurso indicado.

Tipos de autorización en Postman

Postman permite configurar distintos tipos de autorización dependiendo de la seguridad utilizada por una API.

- No Auth: Se utiliza cuando la API no necesita ningún tipo de autenticación.
- API Key: Utiliza una clave única que permite identificar a la aplicación o usuario que realiza la petición.
- Bearer Token: Utiliza un token de acceso que se envía junto con la petición para comprobar que existe autorización.
- Basic Auth: Utiliza un nombre de usuario y una contraseña para autenticar la solicitud.

- Digest Auth: Utiliza un mecanismo de autenticación basado en un desafío y una respuesta, evitando manejar la contraseña exactamente de la misma forma que Basic Auth.
- OAuth 1.0: Permite que una aplicación tenga acceso autorizado a determinados recursos sin utilizar directamente la contraseña del usuario.
- OAuth 2.0: Utiliza tokens para proporcionar acceso a determinados recursos. Es uno de los métodos más utilizados actualmente.
- AWS Signature: Es un método utilizado para autenticar solicitudes realizadas a diferentes servicios de Amazon Web Services.

Conclusion

Las APIs son fundamentales para la comunicación entre diferentes aplicaciones y sistemas. Existen distintos tipos dependiendo de quién puede utilizarlas y diferentes estilos arquitectónicos según las necesidades de cada sistema.

Además, conceptos como request, response, payload y autorización son importantes para comprender cómo se intercambia información de forma organizada y segura.